T2. Modelado

Objetos Sólidos

Modelar un objeto es **darle forma**. Existen dos tipos de modelado:

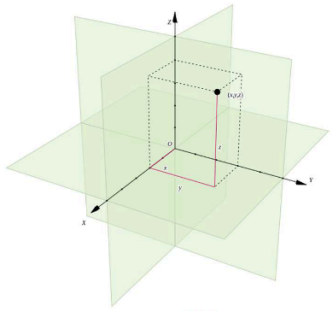
- **Modelado de interior**: se suelen emplear representaciones basadas en **Partición Espacial** que consiste en describir el interior de un objeto diviendolo en regiones pequeñas y sólidas que **no se solapan** entre sí. Por lo general se usan cubos (marching cubes).

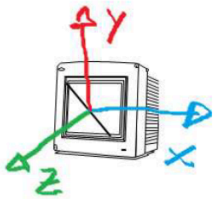
- **Modelado de Exterior** o **Representación de Superficie**: define el contorno del objeto sin hacer referencia a su volumen interior. Proporciona información sobre:
  - **Superficie**: debe dar cuenta su geometría.
  - **Propiedades visuales**: cómo se comporta el color o la textura frente a la luz.
  - Alguna **propiedad física** (como la elasticicidad). Si va a efectuar una simulación física sobre el objeto.

  En la representación de objetos 3D las superficies representadas deben ser **cerradas**.

Ejes coordenados, Puntos y Vectores

Tanto los puntos, como los vectores y los objetos, se representarán sobre los tres **ejes cartesianos** (x es anchura, y es altura, z esprofundidad). Por **defecto**, el rango de valores de las coordenadas va desde **(-1,-1,-1) a (1,1,1)**.





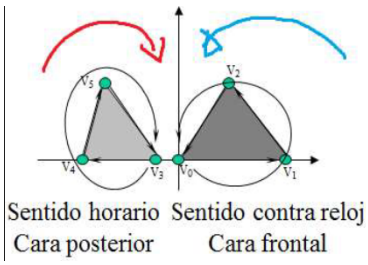
Triangulación

Para modelar cualquier objeto se emplea la **técnica de teselación**, consiste en su representación mediante **polígonos**. Generalmente, se usa el **triángulo**.

$$Número\ de\ triángulos\ =\ Número\ de\ lados\ del\ polígono\ -\ 2.$$

Identificación de Caras

Un **plano** se puede definir por su **vector normal**, que se puede calcular mediante el producto vectorial de dos de sus vectores. Si simplemente se desea conocer la dirección del vector normal, basta aplicar la **regla de la mano derecha** según el recorrido de los vértices. Si se recorren en sentido antihorario la normal será positiva, en caso contrario, será negativa



Nota: para calcular vectores entre 2 puntos, $A(x_1, y_1, z_1)$ y $B(x_2, y_2, z_2)$, su producto $AB = (x_2 - x_1, y_2 - y_1, z_2 - z_1)$

El **producto vectorial**, $u \times v$ en R^3, se calcula como el **determinante** de la matriz 3×3 conformada por los **vectores unitarios** de los ejes, los componentes del vector u y los del v.

$$\begin{vmatrix} i & j & k \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix}$$

Para un punto P(x, y, z);

Ax+By+Cz+D = 0; En la superficie.

Ax+By+Cz+D < 0; está dentro;

Ax+By+Cz+D > 0 esta fuera

La **normal** se puede especificar para un conjunto de vértices con $glNormal3f(x, y, z)$.

En OpenGl se usa $FACE\ CULLING$, es decir sólo se visualizan las **superficies** cuya **normal** se dirige hacia la **cámara**. Por tanto hay que tener en cuenta el **orden** de **creación** de los **vértices**. Está **desactivado por defecto**, pero esto se puede cambiar con $glEnable(GL_CULL_FACE)$ y $glDisable(GL_CULL_FACE)$. Por defecto, se **visualizarán** las **caras frontales** (lo que se puede especificar con $glFrontFace(GL_CCW)$ pero se puede desactivar con $glFrontFace(GL_CW)$.

Un vector normal se puede **normalizar** (es decir, transformar en un vector unitario) **diviendo** cada una de sus componentes por su **módulo**. Esto es útil para procesos de iluminación de objetos, por lo que conviene activar la normalización automática con $glEnable(GL_NORMALIZE)$.

$$v_{normalizaado} = \frac{v}{||v||}$$

Tablas de representación

Para la representación de un objeto en base a polígonos necesitamos conocer, representar y almacenar sus **vértices** y **atributos** en tablas. Tipos de tablas:

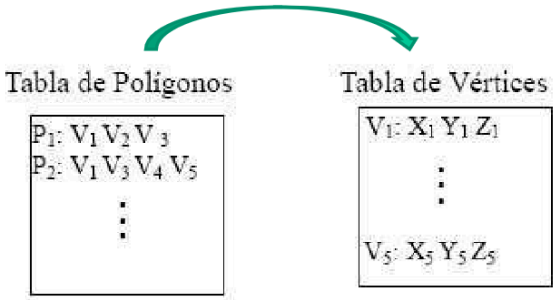
- **Geométricas**: almacenan información sobre los vértices y parámetros sobre los polígonos o superficie que determinan.

- **De atributos**: almacenan información sobre las propiedades físicas de los polígonos (textura, color, transparencia...).

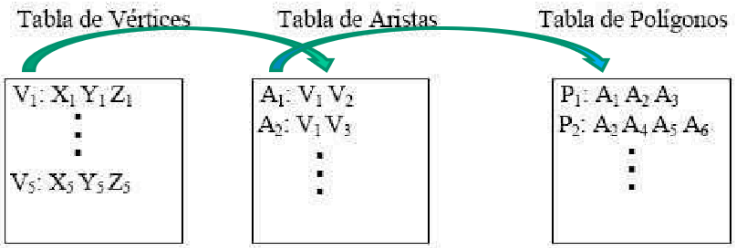
Los tipos de tablas según sus índices pueden ser:

- **Sin indexar**.

- **Indexada**: se simplifica eliminando las aristas utilizando únicamente una **lista** de **vértices** (con 3 coordenadas para cada uno) y una de **polígonos** (con los vértices que definen cada uno). Si un vértice se usa varias veces, se escribe una única vez en la tabla de vértices y se puede **referenciarse varias veces** en la de polígonos.



- **Doblemente indexada**: hay una tabla para los **vértices** (con 3 coordenadas para cada uno), otra para las **aristas** (con dos vértices para cada una y si se desea los dos polígonos que la comparten) y otra para los **polígonos** (con las aristas de cada uno).



Primitivas y ensamblado en OpenGL

En OpenGL, los objetos no se dibujan directamente como figuras complejas, sino a partir de **primitivas geométricas básicas**. Estas primitivas se construyen a partir de **vértices**, y el modo en que se interpretan esos vértices determina la forma final.

La función $glDrawArrays$ permite indicar cómo se deben **interpretar** los vértices almacenados. Por ejemplo, en $glDrawArrays(GL_LINES,\ 0,\ 8)$, se le dice a OpenGL

que dibuje **líneas** usando los vértices desde la **posición 0 hasta 8**. Existen distintos tipos de primitivas:

- GL_POINTS: cada vértice se dibuja como un **punto independiente**.
- GL_LINES: los vértices se agrupan de dos en dos para formar segmentos.
- GL_LINE_STRIP: cada vértice se conecta con el siguiente formando una línea continua.
- GL_LINE_LOOP: igual que el anterior, pero cerrando la figura (el último se conecta con el primero).
- $GL_TRIANGLES$: cada grupo de 3 vértices forma un triángulo independiente.
- $GL_TRIANGLE_STRIP$: cada nuevo vértice forma un triángulo con los dos anteriores.
- $GL_TRIANGLE_FAN$: todos los triángulos comparten un vértice central.
- GL_QUADS: los vérties se toman de cuatro en cuatro para formar cuadriláteros.
- GL_QUADS_STRIP: cada nuevo vértice forma una cuadrilátero con los dos anteriores.
- $GL_POLYGON$: los vértices se usan para construir un polígono convexo.

- <u>Tiras y abanicos de triángulos</u>

Las primitivas como $GL_TRIANGLE_STRIP$ y $GL_TRIANGLE_FAN$ permiten dibujar **múltiples triángulos** de forma eficiente reutilizando vértices.

En $GL_TRIANGLE_STRIP$, el primer triángulo se forma con los tres primeros vértices. A partir de ahí, cada nuevo vértice genera un nuevo triángulo junto con los dos anteriores. El **orden** de los vértices va **alternando** para **mantener** la **orientación** correcta (esto es importante para el descarte de caras).

En $GL_TRIANGLE_FAN$, el primer vértice actúa como centro, y cada nuevo vértice genera un triángulo junto con el anterior y ese vértice central. Es útil para formas tipo abanico o polígonos.

Modos de dibujo de polígonos

La función $glPolygonMode(GLenum\ cara,\ GLenum\ modo)$, el modo especifica como queremos que se pinte la cara del polígono indicada, si rellena con líneas o con puntos para lo cual este parámetro toma los valores:

- GL_FILL: polígonos rellenos (modo normal).
- GL_LINE: solo se dibujan los bordes.
- GL_POINT: solo se dibujan los vértices.

La cara especifica a que cara afectan los cambios, puede tomar los valores: GL_FRONT (en cara frontal), GL_BACK (en cara trasera) o $GL_FRONT_AND_BACK$ (ambas).

Shaders: Vertex y Fragment

En OpenGL moderno, el renderizado se basa en pequeños programas llamados **shaders**, que se ejecutan en la GPU.

- El **Vertex Shader** procesa cada vértice individualmente. Su función principal es transformar la **posición** del vértice a **coordenadas** de pantalla. Recibe atributos como posición, color o normales, y su salida obligatoria es $gl_Position$.

- El **Fragment Shader** se encarga de calcular el **color final** de cada píxel. Recibe información interpolada desde los vértices (como colores) y produce el color final que se verá en pantalla.

- **Envío de vértices a la GPU (VAO y VBO)**

Para dibujar en OpenGL, los datos deben **enviarse** a la GPU usando **buffers**:

- **VBO** (Vertex Buffer Object): almacena los datos de los vértices (posiciones, colores...).
- **VAO** (Vertex Array Object): define cómo se interpretan esos datos.

Primero se crea un VAO y se activa, después se crea un VBO y se cargan los datos. Se define cómo leer esos datos ($glVertexAttribPointer$). Finalmente, se habilitan los atributos ($glEnableVertexAttribArray$).

- **Dibujo con glDrawArrays y glDrawElements**

Hay dos formas principales de dibujar:

- $glDrawArrays$: usa los vértices en orden tal cual están en memoria.
- $glDrawElements$: usa un **array de índices** (EBO, Element Buffer Object, almacena índices que apuntan a los vértices del VBO), lo que permite reutilizar vértices. Esto es más eficiente porque evita duplicar datos.

- **Atributos de vértices**

Un vértice no solo tiene posición, también puede tener: color, coordenadas de textura y normales. Estos datos se **almacenan** juntos en el **VBO** y se **separan** usando $glVertexAttribPointer$. Por ejemplo: ubicación 0 es la posición, ubicación 1 es el color. El **Vertex Shader recibe** estos atributos y puede **pasarlos al Fragment Shader**.

- <u>**Interpolación y paso de datos entre shaders**</u>

Los datos que salen del Vertex Shader (como el color) se **interpolan** automáticamente entre los vértices al rasterizar. Esto significa que: defines **colores en vértices** y OpenGL calcula los colores intermedios en los píxeles. Para ello:

- **out** en vertex shader.
- **in** en fragment shader.